# PYTHON - PART 1

## 1. DEFINITION

Python is a high-level, interpreted, general-purpose programming language.
It was created by Guido van Rossum and first released in 1991.
Python is known for its simple and readable syntax, which makes it easy to learn and use.

## 2. FEATURES

- Simple and easy to learn
- Readable and clean syntax
- Interpreted (no compilation needed)
- Cross-platform (Windows, Linux, macOS)
- Large standard library
- Supports multiple programming paradigms (procedural, OOP, functional)
- Extensive community support

## 3. USES

- Web development (using frameworks like Django, Flask)
- Data analysis and data visualization
- Machine learning and Artificial Intelligence
- Automation and scripting
- Software development
- Testing and debugging
- Game development
- Internet of Things (IoT)

## 4. REAL-WORLD APPLICATIONS

→ Instagram - Uses Python (with Django) for its backend development.
→ YouTube - Uses Python for data processing, recommendation systems, and backend services.
→ Google - Uses Python in many of its internal tools and systems.
→ Netflix - Uses Python for data analysis, machine learning, and automation.
→ Spotify - Uses Python for data analysis and personalized music recommendations.

## 5. A SIMPLE PYTHON PROGRAM

```
# This program prints a message and
# performs a simple calculation.
name = "Ronak"     # string
age = 22           # integer
print("Hello, my name is", name)
print("I am", age, "years old")
print("Sum of 5 + 3 =", 5 + 3)
```

**Output:**

```
Hello, my name is Ronak
I am 22 years old
Sum of 5 + 3 = 8
```

## 6. WHY LEARN PYTHON ?

- Beginner-friendly
- Used in high-demand technologies (AI/ML, Data Science, Web Dev, etc.)
- Large job opportunities
- Versatile and powerful
- Active community and resources
- Helps in problem solving and logical thinking

**Note:**
Python is a great starting point for anyone who wants to build a career in programming, data science or AI. ☺

# PYTHON - PART 2

Next →

## 1. VARIABLES

- A variable is a named location in memory used to store data.
- In Python, we don't need to declare the type of variable explicitly. Python automatically understands the type.
- Variable names must start with a letter or underscore (_), and can contain letters, numbers and underscores.

**Note:**
Variable names are case-sensitive.
age, Age and AGE are different variables.

**Examples:**

```
name    = "Alice"          # string
age     = 20               # integer
height  = 5.6              # float
is_student  = True         # boolean
```

## 2. DATA TYPES

Python has the following built-in data types:

| Data Type | Description | Example |
|---|---|---|
| int | Integer (whole numbers) | 10, -5, 100 |
| float | Floating point numbers (decimal values) | 3.14, -0.001, 2.0 |
| str | String (text) | "Hello", "Python" |
| bool | Boolean (True or False) | True, False |
| list | Ordered collection (mutable) | [1, 2, 3], ['a', 'b'] |
| tuple | Ordered collection (immutable) | (1, 2, 3), ('a', 'b') |
| dict | Key-value pairs (mutable) | {'name': 'Alice', 'age': 20} |
| set | Unordered collection of unique items | {1, 2, 3}, {'a', 'b'} |

## 3. TYPE CHECKING AND TYPE CONVERSION

**Checking type using type()**

```
name = "Python"
print(type(name))   # <class 'str'>
age = 10
print(type(age))    # <class 'int'>
```

**Type conversion (Type Casting)**

- int() → converts to integer
- float() → converts to float
- str() → converts to string
- bool() → converts to boolean

**Examples:**

```
int('10')  → 10
float('3.14') → 3.14
str(100)  → '100'
bool(0)  → False
bool(1)  → True
```

## 4. INPUT AND OUTPUT

**Input** - We use input() to take input from the user.

```
name = input("Enter your name: ")
age  = input("Enter your age: ")
print(name)
print(age)
```

**Output** - We use print() to display output.

```
print("Hello, Python!")
print(10 + 5)    # Output: 15
print("Name:", name)
print(f"Age: {age}")  # f-string (Python 3.6+)
```

## 5. COMMENTS

- Comments are used to add notes to the code.
- They are ignored by the Python interpreter.
- Single-line comment starts with #.
- Multi-line comment uses triple quotes (''' or """).

```
# This is a single-line comment
print("Hello")   # prints Hello
'''
This is a multi-line comment.
It can span multiple lines.
'''
```

## 6. NAMING RULES FOR VARIABLES

- Must start with a letter (a-z, A-Z) or underscore (_)
- Can contain letters, numbers and underscores
- Cannot start with a number
- Cannot use special characters (except underscore)
- Cannot use Python keywords (e.g., if, else, for, while)
- Case-sensitive (age, Age and AGE are different)
- Use meaningful and descriptive names

**Valid Examples:**

name, age, student_name, total2, _count

**Invalid Examples:**

2age, student-name, total@, for, while

# PYTHON – PART 3

Next →

## 1. INPUT AND OUTPUT

- We use `input()` function to take input from the user.
- We use `print()` function to display output.

**Example:**

```
name = input("Enter your name: ")
age  = input("Enter your age: ")
print("Hello", name)
print("You are", age, "years old")
```

## 2. TYPECASTING

- Converting one data type to another.
- Useful when we take input (which is always a string).

| Function | Converts to | Example |
|---|---|---|
| int() | Integer | int('10') → 10 |
| float() | Float | float("3.14") → 3.14 |
| str() | String | str(100) → '100' |
| bool() | Boolean | bool(0) → False<br>bool(1) → True |

## 3. OPERATORS

### A. Arithmetic Operators

| Operator | Name | Example |
|---|---|---|
| + | Add a + b | 5 + 3 = 8 |
| - | Subtract a - b | 5 - 3 = 2 |
| * | Multiply a * b | 5 * 3 = 15 |
| / | Divide a / b | 5 / 2 = 2.5 |
| // | Floor Divide a // b | 5 // 2 = 2 |
| % | Modulus a % b | 5 % 2 = 1 |
| ** | Exponent a ** b | 5 ** 2 = 25 |

### B. Comparison Operators

| Operator | Meaning | Example |
|---|---|---|
| == | Equal to | 5 == 5 → True |
| != | Not equal to | 5 != 3 → True |
| > | Greater than | 5 > 3 → True |
| < | Less than | 5 < 3 → False |
| >= | Greater or equal | 5 >= 5 → True |
| <= | Less or equal | 5 <= 3 → False |

### C. Logical Operators

| Operator | Meaning |
|---|---|
| and | Both conditions True |
| or | At least one True |
| not | Reverses the result |

**Examples:**

True and True → True
True or False → True
not True → False

### D. Assignment Operators

| Operator | Meaning | Example |
|---|---|---|
| = | Assign | x = 5 |
| += | Add and assign | x += 2 (x = x + 2) |
| -= | Subtract and assign | x -= 2 |
| *= | Multiply and assign | x *= 2 |
| /= | Divide and assign | x //= 2 |
| //= | Floor divide and assign | x //= 2 |
| %= | Modulus and assign | x %= 2 |
| **= | Exponent and assign | x **= 2 |

### E. Membership Operators

| Operator | Meaning | Example |
|---|---|---|
| in | Returns True if value is in a sequence | 'a' in 'cat' → True<br>3 in [1, 2, 3] → True |
| not in | Returns True if value is not in a sequence | 'z' not in 'cat' → True<br>4 not in [1, 2, 3] → True |

### F. Identity Operators

| Operator | Meaning | Example |
|---|---|---|
| is | Returns True if both variables point to the same object | a is b |
| is not | Returns True if variables do not point to the same object | a is not b |

## 4. CONDITIONAL STATEMENTS

- Used to make decisions and execute code based on conditions.

### 1. if Statement

```
age = 18
if age >= 18:
  print("You are eligible")
```

- Executes the block if condition is True.

### 2. if-else Statement

```
age = 16
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

- Executes if block if True, otherwise else block.

### 3. if-elif-else Statement

```
marks = 85
if marks >= 90:
    print("A Grade")
elif marks >= 70:
    print("B Grade")
else:
    print("C Grade")
```

- Checks multiple conditions one by one.

**Common Comparison Operators**

| | |
|---|---|
| == | Equal to |
| != | Not equal to |
| > | Greater than |
| < | Less than |
| >= | Greater or equal to |
| <= | Less or equal to |

## 5. NESTED IF STATEMENT

```
age = 20
if age >= 18:
    if age >= 60:
        print("Senior")
    else:
        print("Adult")
else:
    print("Minor")
```

- An if statement inside another if statement.

## 6. TERNARY OPERATOR (SHORT IF-ELSE)

- Used for simple conditions in one line.

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)  # Output: Adult
```

# PYTHON – PART 4

## 1. LOOPS IN PYTHON

- Loops are used to repeat a block of code multiple times.
- They help us execute a set of statements again and again.
- Python supports two main loops: while loop and for loop.

## 2. WHILE LOOP

- The while loop repeats a block of code as long as the condition is True.
- It is used when we don't know how many times the loop will run.

Syntax:

```
while condition:
    # block of code
```

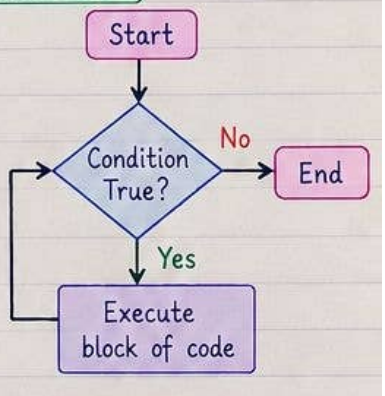
Example:

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

Output:

```
1
2
3
4
5
```

Flowchart:




## 3. FOR LOOP

- The for loop is used to iterate over a sequence like list, tuple, string, range, etc.
- It is used when we know in advance how many times the loop should run.

Syntax:

```
for variable in sequence:
    # block of code
```

Example 1: Using range()

```
for i in range(1, 6):
    print(i)
```

Output:

```
1
2
3
4
5
```

Example 2: Iterating over a list

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

## 4. LOOP CONTROL STATEMENTS

These statements are used to change the flow of a loop.

### 1. break

- Terminates the loop immediately.
- Used when a certain condition is met.

Example:

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

Output:

```
1
2
```

### 2. continue

- Skips the current iteration and continues with the next.

Example:

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Output:

```
1
2
4
5
```

### 3. pass

- Does nothing (used as a placeholder).
- Useful when we will write the code later.

Example:

```
for i in range(3):
    if i == 1:
        pass
    print(i)
```

Output:

```
0
1
2
```

## 5. NESTED LOOPS

- A loop inside another loop is called a nested loop.

Example:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j)
```

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

## 6. KEY POINTS

- ✓ while loop: repeats while condition is True.
- ✓ for loop: iterates over a sequence.
- ✓ Use while loop when iterations are unknown.
- ✓ Use for loop when iterations are known.
- ✓ break: terminates the loop.
- ✓ continue: skips current iteration.
- ✓ pass: does nothing.
- ✓ Loops are widely used in real-world programs.

# PYTHON – PART 5

## 1. STRINGS (Python)

- A string is a sequence of characters enclosed in quotes.
- Strings are immutable (cannot be changed).
- They can contain letters, numbers, symbols and spaces.

Examples:

```
s1 = "Hello"
s2 = 'Python'
s3 = "123"
s4 = 'A'
```

## 2. INDEXING

- Each character in a string has an index.

```
s = "Python"
P  y  t  h  o  n   ← Positive index
0  1  2  3  4  5
-6 -5 -4 -3 -2 -1  ← Negative index
```

## 3. SLICING

- Used to get a substring from a string.

Examples:

```
s = "Python"
s[0:2]   # Py
s[2:5]   # tho
s[:3]    # Pyt
s[3:]    # hon
s[-3:]   # hon
s[::-1]  # nohtyP (Reverse)
```

Note:
s [start:end]
(end not included)
Default start = 0
Default end = len(s)

## 4. STRING OPERATIONS

| Operation | Description | Example (s = "Python") | Result |
|---|---|---|---|
| len(s) | Length of string | len(s) | 6 |
| + | Concatenation | s + " is great" | "Python is great" |
| * | Repetition | s * 2 | "PythonPython" |
| in | Membership | 'P' in s | True |
| not in | Non-membership | 'z' not in s | True |

## 5. COMMON STRING METHODS

| Method | Description | Example (s = "Python") | Result |
|---|---|---|---|
| lower() | Convert to lowercase | s.lower() | " python " |
| upper() | Convert to uppercase | s.upper() | " PYTHON " |
| strip() | Remove spaces (both ends) | s.strip() | " Python" |
| lstrip() | Remove left spaces | s.lstrip() | " Python " |
| rstrip() | Remove right spaces | s.rstrip() | " Python" |
| replace() | Replace substring | s.replace("P","J") | " Jython " |
| split() | Split into list | "a,b,c".split(",") | ['a','b','c'] |
| join() | Join elements | "-".join(['a','b','c"]) | "a-b-c" |

## 6. ESCAPE SEQUENCES

| Sequence | Meaning |
|---|---|
| \n | New line |
| \t | Tab space |
| \" | Double quote |
| \' | Single quote |
| \\ | Backslash |

Example:

```
s = "Hello\nWorld"
print(s)
```

Output:
Hello
World

## 7. STRING FORMATTING

### A. f-string (Python 3.6+)

- Simplest and most used method.

```
name = "Ronak"
age = 22
print(f"My name is {name}")
print(f"I am {age} years old")
```

Output:
My name is Ronak
I am 22 years old

### B. format() method

- Older method using placeholders.

```
name = "Ronak"
age = 22
print("My name is {}".format(name))
print("I am {} years old".format(age))
```

Output:
My name is Ronak
I am 22 years old

### C. % formatting (older)

- Old style formatting (still used).

```
name = "Ronak"
age = 22
print("My name is %s" % name)
print("I am %d years old" % age)
```

Output:
My name is Ronak
I am 22 years old

## 8. RAW STRINGS

- Used to ignore escape characters.
- Useful when working with file paths.

```
path = r"C:\Users\Ronak\Desktop"
print(path)   # C:\Users\Ronak\Desktop
```

## 9. KEY POINTS

- ✔ Strings are sequences of characters.
- ✔ Strings are immutable.
- ✔ We can access characters using positive and negative indexing.
- ✔ Use slicing to get a part of a string.
- ✔ Python provides many useful string methods.
- ✔ Strings can be formatted using f-strings, format() or %.
- ✔ Escape sequences help include special characters.
- ✔ Raw strings are useful for file paths and regex.

# PYTHON - PART 6

Next →

## 1. LIST ([])

- A list is an ordered, changeable (mutable) collection of items.
- It can contain duplicate values.
- It can store different data types.
- We use square brackets [] to create a list.

**Example:**

```
fruits = ["apple", "banana", "cherry", 10, True]
print(fruits)
print(fruits[0])   # first element
print(fruits[-1])  # last element
```

**Output:**

```
['apple', 'banana', 'cherry', 10, True]
apple
True
```

**Common List Operations:**

| Operation | Description | Example |
|---|---|---|
| Access element | Get element by index | fruits[0] → 'apple' |
| Negative index | Access from end | fruits[-1] → True |
| Slicing | Get a sublist | fruits[1:4] → ['banana', 'cherry', 10] |
| Append | Add element at end | fruits.append("mango") |
| Insert | Add element at index | fruits.insert(1, 'orange") |
| Remove | Remove first occurrence | fruits.remove('banana") |
| Pop | Remove and return element | fruits.pop() |
| Sort | Sort the list | nums.sort() |
| Length | Get number of elements | len(fruits) |
| Clear | Remove all elements | fruits.clear() |

**More Examples:**

```
nums = [1, 2, 3, 4]
nums.append(5)
nums.insert(1, 10)
nums.remove(3)
print(nums)
print(len(nums))
```

**Output:**

```
[1, 10, 2, 4, 5]
5
```

**List Comprehension:**

```
squares = [x*x for x in range(5)]
print(squares)
```

**Output:**

```
[0, 1, 4, 9, 16]
```

## 2. TUPLE (())

- A tuple is an ordered, unchangeable (immutable) collection of items.
- It can contain duplicate values.
- We use parentheses () to create a tuple.
- Tuples are faster and safer for read-only data.

**Example:**

```
coordinates = (10.5, 20.3, "Delhi")
print(coordinates)
print(coordinates[0])
```

**Output:**

```
(10.5, 20.3, 'Delhi')
10.5
```

**Common Tuple Operations:**

| Operation | Description | Example |
|---|---|---|
| Access element | Get element by index | t[0] → 10.5 |
| Count | Count occurrences | t.count('Delhi") |
| Index | Find index | t.index(20.3) |
| Length | Get number of elements | len(t) |

**Tuple Packing and Unpacking:**

```
# Packing
t = (1, 2, 3)
# Unpacking
a, b, c = t
print(a, b, c)
```

**Output:**

```
1 2 3
```

## 3. SET ({})

- A set is an unordered, changeable (mutable) collection of unique items.
- It does not allow duplicate values.
- We use curly braces {} to create a set.
- Useful for finding unique values and set operations.

**Example:**

```
numbers = {1, 2, 3, 4, 4, 5}
print(numbers)
# duplicates are removed
print(2 in numbers)
```

**Output:**

```
{1, 2, 3, 4, 5}
True
```

**Common Set Operations:**

| Operation | Description | Example (A = {1,2,3}, B = {3,4,5}) |
|---|---|---|
| Add | Add element | A.add(6) → {1,2,3,6} |
| Remove | Remove element | A.remove(2) → {1,3} |
| Union | All unique elements | A \| B → {1,2,3,4,5} |
| Intersection | Common elements | A & B → {3} |
| Difference | Elements in A not in B | A - B → {1,2} |
| Length | Number of elements | len(A) |

**Example:**

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B)  # Union
print(A & B)  # Intersection
print(A - B)  # Difference
```

**Output:**

```
{1, 2, 3, 4, 5}
{3}
{1, 2}
```

## 4. DICTIONARY ({key: value})

- A dictionary is an ordered (Python 3.7+), changeable collection of key-value pairs.
- Keys must be unique and immutable (string, number, tuple).
- Values can be of any data type.
- We use curly braces {} with key: value pairs to create a dictionary.

**Example:**

```
student = {
    "name": "Rahul",
    "age": 20,
    "course": "Python"
}
print(student)
print(student["name"])
```

**Output:**

```
{'name': 'Rahul', 'age': 20, 'course': 'Python'}
Rahul
```

**Common Dictionary Operations:**

| Operation | Description | Example |
|---|---|---|
| Access value | Get value by key | student["name"] → 'Rahul' |
| Add/Update | Add new or update value | student["age"] = 21 |
| Remove | Remove a key-value pair | student.pop("course") |
| Keys | Get all keys | student.keys() |
| Values | Get all values | student.values() |
| Items | Get key-value pairs | student.items() |
| Length | Get number of items | len(student) |

**More Examples:**

```
student["city"] = "Udaipur"
print(student.keys())
print(student.values())
print(student.items())
```

**Output:**

```
dict_keys(['name', 'age', 'course', 'city'])
dict_values(['Rahul', 20, 'Python', 'Udaipur'])
dict_items([('name', 'Rahul'), ('age', 20), ('course', 'Python'), ('city', 'Udaipur')])
```

**Key Points:**

- ✔ List → ordered, mutable, allows duplicates, uses []
- ✔ Tuple → ordered, immutable, allows duplicates, uses ()
- ✔ Set → unordered, mutable, unique items, uses {}
- ✔ Dictionary → ordered (3.7+), mutable, key-value pairs, uses {}

# PYTHON - PART 7

## 1. FUNCTIONS

- A function is a block of reusable code that performs a specific task.
- Helps in code reusability and modularity.
- We define a function using `def` keyword.
- A function can take parameters and can also return a value.

**Syntax:**

```
def function_name(parameters):
    # block of code
    return value
```

**Example:**

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)  # Output: 8
```

## 2. TYPES OF FUNCTIONS

| Type | Description |
|---|---|
| Built-in functions | Provided by Python (e.g., print(), len()) |
| User-defined functions | Created by the user |
| Lambda functions | Anonymous (small) functions |
| Recursive functions | Functions that call themselves |

**Examples:**

```
print("Hello")                          # Built-in
def greet(): ...                        # User-defined
square = lambda x: x*x                  # Lambda
def fact(n): return n * fact(n-1)       # Recursive
```

## 3. FUNCTION ARGUMENTS

| Type | Description | Example |
|---|---|---|
| Positional | Passed in correct order | def add(a, b)<br>add(5, 3) |
| Keyword | Passed with parameter name | add(a=5, b=3) |
| Default | Has a default value | def greet(name="User") |
| Variable length (*args) | Accepts any number of positional arguments | def sum_all(*args) |
| Variable length (**kwargs) | Accepts any number of keyword arguments | def show(**kwargs) |

## 4. RETURN VALUES

- A function can return a value using `return` keyword.

**Example:**

```
def square(x):
    return x * x

print(square(4))  # Output: 16
```

**Multiple Returns:**

```
def get_values():
    return 10, 20, 30
a, b, c = get_values()
print(a, b, c)  # 10 20 30
```

**Note:**

- A function can return multiple values (as tuple).
- If no return is used, it returns None.

## 5. LAMBDA FUNCTIONS

- A small anonymous function defined using `lambda` keyword.
- Mostly used for short, simple operations.

**Syntax:**

```
lambda arguments: expression
```

**Example:**

```
square = lambda x: x*x
print(square(4))  # 16
```

## 6. SCOPE OF VARIABLES

- Variables defined inside a function are local to that function.
- Variables defined outside are global.
- Use `global` keyword to modify a global variable inside a function.

**Example:**

```
x = 10    # global variable
def change():
    global x
    x = 20
change()
print(x)    # 20
```

**Note:**

- Local variables exist only inside the function.
- Global variables can be accessed anywhere.
- Use `global` only when necessary.

## 7. DOCSTRING

- A docstring is a string used to document a function.
- It is written inside triple quotes (""" ").
- Helps in explaining what the function does.

**Example:**

```
def add(a, b):
    """This function returns
    the sum of two numbers."""
    return a + b
print(add.__doc__)
```

**Output:**

```
This function returns
the sum of two
numbers.
```

## 8. MODULES

- A module is a file containing Python code (functions, classes, variables) that can be used in other programs.
- We can import modules to reuse code.

**Example:**

```
import module_name
# or
from module_name import name
# or
from module_name import *
```

**Example:**

```
import math
print(math.sqrt(16))  # 4.0
from math import pi
print(pi)  # 3.14159
```

## 9. COMMON BUILT-IN MODULES

| Module | Description | Example |
|---|---|---|
| math | Mathematical functions | math.sqrt(25) |
| random | Generate random values | random.randint(1, 10) |
| os | Operating system functions | os.getcwd() |
| datetime | Work with dates and time | datetime.now() |
| sys | System related functions | sys.version |

## 10. KEY POINTS

- ✔ Functions help in code reusability.
- ✔ Use def to define a function.
- ✔ A function can take arguments and return a value.
- ✔ Lambda functions are useful for short tasks.
- ✔ Variables inside a function are local by default.
- ✔ Modules help in organizing code into separate files.
- ✔ Use docstrings to document your functions.
- ✔ Python has many useful built-in modules.

# PYTHON - PART 8

## 1. EXCEPTION HANDLING

- Used to handle runtime errors and prevent program crash.
- Helps in graceful error handling.
- We use try, except, else and finally blocks.

**Syntax:**

```
try:
    # code that may cause error
except ExceptionType:
    # handle the exception
else:
    # runs if no exception
finally:
    # always runs
```

**Example:**

```
try:
    a = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("No error")
finally:
    print("Execution completed")
```

**Common Exceptions:**

| Exception | Meaning |
|---|---|
| ZeroDivisionError | Division by zero |
| ValueError | Invalid value |
| TypeError | Wrong type |
| FileNotFoundError | File not found |
| IndexError | Index out of range |
| KeyError | Key not found in dictionary |
| AttributeError | Attribute does not exist |

## 2. FILE HANDLING

- Files are used to store data permanently on disk.
- We use open() function to work with files.
- Modes: 'r' (read), 'w' (write), 'a' (append), 'b' (binary).

**Syntax:**

```
f = open("file.txt", "mode")
```

**File Modes:**

| | |
|---|---|
| 'r' | Read (default) |
| 'w' | Write (overwrites) |
| 'a' | Append (adds at end) |
| 'b' | Binary mode |
| 'r+' | Read and write |

**Common File Operations:**

| Method | Description | Example |
|---|---|---|
| read() | Read entire file | f.read() |
| readline() | Read one line | f.readline() |
| readlines() | Read all lines | f.readlines() |
| write(data) | Write data to file | f.write("Hello") |
| writelines(list) | Write list to file | f.writelines(lines) |
| close() | Close the file | f.close() |

**Example 1: Writing to a file**

```
f = open("demo.txt", "w")
f.write("Hello World\n")
f.close()
```

**Example 2: Reading from a file**

```
f = open("demo.txt", "r")
data = f.read()
print(data)
f.close()
```

## 3. WITH STATEMENT (File Handling)

- Automatically closes the file after use.
- No need to call close() explicitly.
- Recommended way to handle files.

**Example:**

```
with open("demo.txt", "r") as f:
    data = f.read()
    print(data)
# File is automatically closed
```

## 4. READ & WRITE EXAMPLE (Append Mode)

**Example:**

```
with open("notes.txt", "a") as f:
    f.write("Python is awesome!\n")

with open("notes.txt", "r") as f:
    print(f.read())
```

## 5. WORKING WITH FILE PATHS

- We can work with file and folder paths using the os module.

**Example:**

```
import os
print(os.getcwd())  # current directory
os.mkdir("data")    # create folder
os.listdir()        # list files/folders
```

## 6. HANDLING MULTIPLE EXCEPTIONS

**Example:**

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
except ValueError:
    print("Invalid input")
except ZeroDivisionError:
    print("Cannot divide by zero")
except Exception as e:
    print("Other error:", e)
```

## 7. RAISING EXCEPTIONS

- We can raise our own exceptions using raise keyword.

**Example:**

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative")
```

## 8. TRY WITH ELSE AND FINALLY

**Example:**

```
try:
    n = int(input("Enter a number: "))
    print("Number:", n)
except ValueError:
    print("Invalid input")
else:
    print("No exception occurred")
finally:
    print("This will always run")
```

## 9. KEY POINTS

- ✔ Exceptions help in graceful error handling.
- ✔ Use try, except, else, finally blocks.
- ✔ Use specific exceptions instead of general Exception.
- ✔ Always close files (use with statement).
- ✔ Files can be read, written and appended.
- ✔ Use os module for file and folder operations.

# PYTHON - PART 9

## 1. OOPS BASICS

- OOPs (Object Oriented Programming) helps us structure code using objects and classes.
- It makes code modular, reusable and easy to maintain.
- Main concepts: Class, Object, Encapsulation, Inheritance, Polymorphism and Abstraction.

## 2. CLASS AND OBJECT

- A class is a blueprint for creating objects.
- An object is an instance of a class.
- We define a class using the class keyword.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def show(self):
        print(f"Name: {self.name}, Age: {self.age}")

s1 = Student("Ronak", 22)
s1.show()
```

Output:

```
Name: Ronak, Age: 22
```

## 3. ENCAPSULATION

- Binding data and methods together and restricting direct access to data.
- We use _name for protected members and __name for private members.

Example:

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # private
    def get_balance(self):
        return self.__balance

acc = BankAccount(1000)
print(acc.get_balance())
```

Output:

```
1000
```

## 4. INHERITANCE

- A child class inherits properties and methods from a parent class.
- It helps in code reusability.

Example:

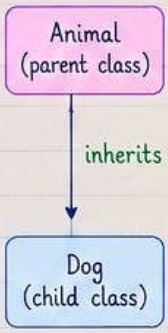
```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

d = Dog()
d.speak()
```

Output:

```
Dog barks
```




## 5. POLYMORPHISM

- Same method name, different behavior in different classes.
- Achieved using method overriding.

Example:

```
class Cat:
    def sound(self):
        print("Meow")

class Dog:
    def sound(self):
        print("Woof")

for animal in (Cat(), Dog()):
    animal.sound()
```

Output:

```
Meow
Woof
```

## 6. ABSTRACTION

- Hiding internal complexity and showing only essential features.
- Achieved using abstract class and abstract methods.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r * self.r

c = Circle(5)
print(c.area())
```

Output:

```
78.5
```

## 7. SPECIAL METHODS

- Also called dunder methods.
- They start and end with double underscores (__).
- Used to define special behavior for objects.

```
class Book:
    def __init__(self, title):
        self.title = title
    def __str__(self):
        return f"Book: {self.title}"

b = Book("Python")
print(b)
```

Output:

```
Book: Python
```

## 8. TYPES OF METHODS

| Method Type | Syntax | Description |
|---|---|---|
| Instance Method | def method(self): | Works with instance variables (self). |
| Class Method | @classmethod<br>def method(cls): | Works with class variables (cls). |
| Static Method | @staticmethod<br>def method(): | Does not access instance or class variables. |

Example:

```
class MathUtil:
    @staticmethod
    def add(a, b):
        return a + b

print(MathUtil.add(4, 5))
```

Output:

```
9
```

## 9. KEY POINTS

- ✔ Class is a blueprint, object is an instance.
- ✔ Encapsulation protects data.
- ✔ Inheritance allows code reusability.
- ✔ Polymorphism provides different behavior.
- ✔ Abstraction hides implementation details.
- ✔ Use __init__() constructor to initialize objects.
- ✔ Use self to access instance variables.
- ✔ OOPs makes code modular, reusable and easier to maintain.
- ✔ Special methods (dunder methods) define object behavior.
- ✔ Use @classmethod and @staticmethod when needed.

# PYTHON - PART 10

## 1. LAMBDA FUNCTIONS

- A lambda function is a small, anonymous function defined using lambda keyword.
- It can take any number of arguments. but can have only one expression.
- Useful for short operations.

**Syntax:**

```
lambda arguments: expression
```

**Example:**

```
square = lambda x: x * x
print(square(5))
```

**Output:**

```
25
```

## 2. LIST COMPREHENSION

- A concise way to create lists.
- More readable and faster than using loops.
- Can also include conditions.

**Syntax:**

```
[expression for item in iterable if condition]
```

**Example 1: Square of numbers**

```
squares = [x * x for x in range(5)]
print(squares)
```

**Output:**

```
[0, 1, 4, 9, 16]
```

**Example 2: With condition**

```
even = [x for x in range(10) if x % 2 == 0]
print(even)
```

**Output:**

```
[0, 2, 4, 6, 8]
```

## 3. map() FUNCTION

- Applies a function to all items in an iterable.
- Returns a map object (need to convert to list).

**Syntax:**

```
map(function, iterable)
```

**Example:**

```
nums = [1, 2, 3, 4]
result = list(map(lambda x: x*2, nums))
print(result)
```

**Output:**

```
[2, 4, 6, 8]
```

## 4. filter() FUNCTION

- Filters items from an iterable based on a condition.
- Returns a filter object (need to convert to list).

**Syntax:**

```
filter(function, iterable)
```

**Example:**

```
nums = [1, 2, 3, 4, 5]
result = list(filter(lambda x: x % 2 == 0, nums))
print(result)
```

**Output:**

```
[2, 4]
```

## 5. reduce() FUNCTION

- Performs a rolling computation on iterable.
- Available in functools module.

**Syntax:**

```
from functools import reduce
reduce(function, iterable)
```

**Example:**

```
from functools import reduce
nums = [1, 2, 3, 4]
result = reduce(lambda x, y: x + y, nums)
print(result)
```

**Output:**

```
10
```

## 6. ITERATORS

- An iterator is an object that allows us to iterate over a collection one element at a time.
- Uses iter() and next() functions.

**Example:**

```
nums = [10, 20, 30]
it = iter(nums)
print(next(it))   # 10
print(next(it))   # 20
print(next(it))   # 30
```

**Output:**

```
10
20
30
```

**Note:**
When no elements are left, StopIteration exception is raised.

## 7. GENERATORS AND YIELD

- A generator is a special type of iterator.
- It uses yield keyword to return values one at a time.
- More memory efficient than normal functions.

**Example:**

```
def count(n):
    for i in range(n):
        yield i

for num in count(5):
    print(num)
```

**Output:**

```
0
1
2
3
4
```

## 8. DECORATORS

- A decorator is a function that modifies the behavior of another function.
- Used for logging, authentication, timing etc.

**Example:**

```
def decorator(func):
    def wrapper():
        print("Before function")
        func()
        print("After function")
    return wrapper
@decorator
def greet():
    print("Hello!")
greet()
```

**Output:**

```
Before function
Hello!
After function
```

## 9. *args and **kwargs

- *args allows passing variable number of positional arguments.
- **kwargs allows passing variable number of keyword arguments.

**Example:**

```
def demo(*args, **kwargs):
    print("args:", args)
    print("kwargs:", kwargs )
demo(1, 2, 3, name="Ronak", age=22)
```

**Output:**

```
args: (1, 2, 3)
kwargs: {'name': 'Ronak',
         'age': 22}
```

## 10. KEY POINTS

- ✔ Lambda functions are small anonymous functions.
- ✔ List comprehension creates lists in a concise way.
- ✔ map() applies a function to all items.
- ✔ filter() filters items based on a condition.
- ✔ reduce() performs rolling computation (from functools).
- ✔ Iterators return elements one at a time.
- ✔ Generators use yield and are memory efficient.
- ✔ Decorators modify the behavior of functions.
- ✔ *args and **kwargs allow variable arguments.

# PYTHON - PART 11

## 1. MODULES AND PACKAGES

- A module is a file containing Python code.
- A package is a folder containing multiple modules.
- Helps in code reusability and organization.
- We can create our own modules and packages.

**Example:**

```
mypackage/
    __init__.py
    math_utils.py
    string_utils.py
main.py
```

```
from mypackage import math_utils
print(math_utils.add(2, 3))
```

**Output:**

```
5
```

## 2. COMMONLY USED MODULES

| Module | Description |
|---|---|
| math | Mathematical functions |
| random | Generate random values |
| datetime | Work with date and time |
| os | Operating system functions |
| sys | System specific parameters |
| re | Regular expressions |
| json | Work with JSON data |
| csv | Read and write CSV files |
| time | Time related functions |
| collections | Specialized data structures |

## 3. MATH MODULE

- Provides mathematical functions and constants.
- Useful for numerical calculations.

**Example:**

```
import math

print(math.sqrt(25))      # 5.0
print(math.factorial(5))  # 120
print(math.pi)            # 3.14159
print(math.sin(math.pi/2))  # 1.0
```

**Output:**

```
5.0
120
3.14159
1.0
```

## 4. RANDOM MODULE

- Used to generate random numbers, choices and sequences.
- Useful for simulations and testing.

**Example:**

```
import random
print(random.randint(1, 10))  # random int
print(random.random())        # random float
print(random.choice([1, 2, 3]))
print(random.sample([1, 2, 3, 4], 2))
```

**Output:**

```
7
0.4162
2
[3, 1]
```

## 5. DATETIME MODULE

- Used to work with dates and time.
- Useful for logging, timestamps and time calculations.

**Example:**

```
from datetime import datetime, timedelta
now = datetime.now()
print(now)
print(now.strftime("%Y-%m-%d"))
new_date = now + timedelta(days=5)
print(new_date.strftime("%Y-%m-%d"))
```

**Output:**

```
2026-10-01 14:30:25.123456

2026-10-01
2026-10-06
```

## 6. OS MODULE

- Provides functions to interact with operating system.
- Useful for file and directory operations.

**Example:**

```
import os
print(os.getcwd())  # current directory
os.mkdir("data")    # create folder
print(os.listdir())  # list files/folders
```

**Output:**

```
C:\Users\Ronak
['data', 'main.py']
```

## 7. SYS MODULE

- Provides access to system specific parameters.
- Useful for command line arguments and exits.

**Example:**

```
import sys
print(sys.version)
print(sys.platform)
print(sys.argv)  # command line arguments
```

**Output:**

```
3.11.5 (main, ...)
win32
['main.py"]
```

## 8. JSON MODULE

- Used to work with JSON data (read and write).
- Useful for APIs and data storage.

**Example:**

```
import json
data = {"name": "Ronak", "age": 22}
json_str = json.dumps(data)
print(json_str)
new_data = json.loads(json_str)
print(new_data["name"])
```

**Output:**

```
{"name": "Ronak",
"age": 22}

Ronak
```

## 9. REGULAR EXPRESSIONS (re)

- Used to search, match and manipulate text patterns.
- Useful for data cleaning and text processing.

**Example:**

```
import re
text = "My email is ronak@gmail.com"
match = re.search(r"\w+@\w+\.\w+", text)
print(match.group())
emails = re.findall(r"\w+@\w+\.\w+", text)
print(emails)
```

**Output:**

```
ronak@gmail.com

['ronak@gmail.com"]
```

## 10. KEY POINTS

- ✓ Modules help in code reusability.
- ✓ Packages contain multiple modules.
- ✓ Use built-in modules like math, random, datetime, os, sys.
- ✓ json is useful for APIs and data storage.
- ✓ re helps in text processing.
- ✓ We can create our own modules and packages.
- ✓ Modules make code modular and easy to maintain.
- ✓ Import only required modules.
- ✓ Use alias (import module as alias) for readability.

# PYTHON – PART 12

## 1. LIST, TUPLE, SET, DICTIONARY (REVISION)

- Different data types to store multiple values.
- Each has different properties and use cases.

| Type | Syntax | Properties | Example |
|---|---|---|---|
| List | [ ] | Ordered, Mutable, Allows duplicate | [1, 2, 3] |
| Tuple | ( ) | Ordered, Immutable, Allows duplicate | (1, 2, 3) |
| Set | { } | Unordered, Mutable, No duplicate | {1, 2, 3} |
| Dictionary | {key: value} | Ordered (Python 3.7+), Mutable, Unique keys | {'a': 1, 'b': 2} |

## 2. DICTIONARY COMPREHENSION

- A concise way to create dictionaries.
- Can also include conditions.

Syntax:

```
{key: value for item in iterable
            if condition}
```

Example:

```
squares = {x: x*x for x in range(5)}
print(squares)
```

Output:

```
{0: 0, 1: 1,
2: 4, 3: 9,
4: 16}
```

Example with condition:

```
even_sq = {x: x*x for x in range(10)
           if x % 2 == 0}
print(even_sq)
```

Output:

```
{0: 0, 2: 4,
4: 16, 6: 36,
8: 64}
```

## 3. SET COMPREHENSION

- A concise way to create sets.
- Automatically removes duplicates.

Syntax:

```
{expression for item in iterable if condition}
```

Example:

```
s = {x for x in range(10) if x % 2 == 0}
print(s)
```

Output:

```
{0, 2, 4,
6, 8}
```

## 4. ZIP() FUNCTION

- Used to combine multiple iterables.
- Returns an iterator of tuples.

Example:

```
names = ["Ronak", "Aman", "Nikhil"]
ages = [22, 21, 23]
result = list(zip(names, ages))
print(result)
```

Output:

```
[('Ronak', 22),
('Aman', 21),
('Nikhil', 23)]
```

## 5. ENUMERATE() FUNCTION

- Adds a counter to an iterable.
- Returns index and value.

Example:

```
fruits = ["Apple", "Banana", "Mango"]
for index, value in enumerate(fruits):
    print(index, value)
```

Output:

```
0 Apple
1 Banana
2 Mango
```

## 6. UNPACKING OPERATORS (*, **)

- * used for unpacking iterables.
- ** used for unpacking dictionaries.

Example (* for list/tuple):

```
nums = [1, 2, 3]
print(*nums)          # unpack list
print(max(*nums))     # pass as args
```

Output:

```
1 2 3
3
```

Example (** for dict):

```
def show(a, b):
    print(a, b)
data = {'a': 10, 'b': 20}
show(**data)
```

Output:

```
10 20
```

## 7. LAMBDA WITH HIGHER ORDER FUNCTIONS

- Using lambda with map(), filter(), reduce(), sorted() etc.
- Useful for short and simple operations.

Example:

```
nums = [1, 2, 3, 4]
squared = list(map(lambda x: x*x, nums))
even = list(filter(lambda x: x % 2 == 0, nums))
from functools import reduce
sum_all = reduce(lambda x, y: x + y, nums)
print(squared)
print(even)
print(sum_all)
```

Output:

```
[1, 4, 9, 16]
[2, 4]
10
```

## 8. SORTED() FUNCTION

- Returns a new sorted list (does not modify original).
- Can sort in ascending or descending order.

Example:

```
nums = [5, 1, 4, 2, 3]
print(sorted(nums))                  # ascending
print(sorted(nums, reverse=True)) # descending

words = ["apple", "banana", "cherry"]
print(sorted(words))
```

Output:

```
[1, 2, 3, 4, 5]
[5, 4, 3, 2, 1]

['apple', 'banana',
'cherry"]
```

## 9. ANY() AND ALL() FUNCTIONS

- any() returns True if any element is True.
- all() returns True if all elements are True.

Example:

```
nums1 = [0, 0, 1, 0]
nums2 = [1, 2, 3]
print(any(nums1))     # True
print(all(nums1))     # False
print(all(nums2))     # True
```

Output:

```
True
False
True
```

## 10. KEY POINTS

- ✓ List, tuple, set, dictionary are important data structures.
- ✓ Dictionary and set comprehension make code concise.
- ✓ zip() combines multiple iterables.
- ✓ enumerate() gives index and value.
- ✓ * and ** are unpacking operators.
- ✓ lambda works well with higher order functions.
- ✓ sorted() returns a new sorted list.
- ✓ any() checks if any element is True.
- ✓ all() checks if all elements are True.
- ✓ These concepts make code clean, short and efficient.

# PYTHON - PART 13

## 1. FILE AND DIRECTORY HANDLING (ADVANCED)

- We can work with files and directories using os and pathlib.
- Useful for data processing, automation and ML projects.
- Operations: create, read, write, append, delete, rename.
- We can also work with folders and list files.

**Example 1: Working with folders and files**

```
import os
os.mkdir("data")          # create folder
with open("data/sample.txt", "w") as f:
    f.write("Hello Python")
files = os.listdir("data")    # list files
print(files)
```

**Output:**

```
['sample.txt']
```

**Example 2: Using pathlib (modern way)**

```
from pathlib import Path
path = Path("project")
path.mkdir(exist_ok=True)
file = path / "info.txt"
file.write_text("Python is fun")
print(file.read_text())
```

**Output:**

```
Python is fun
```

## 2. WORKING WITH CSV FILES

- CSV (Comma Separated Values) files are widely used to store tabular data.
- We can read, write and process CSV files using csv module or pandas.

**Example: Read and write CSV using csv module**

```
import csv
# Write to CSV
data = [["Name", "Age"], ["Ronak", 22], ["Aman", 21]]
with open("people.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data)
# Read from CSV
with open("people.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

**Output:**

```
['Name', 'Age']
['Ronak', '22']
['Aman', '21']
```

## 3. WORKING WITH JSON FILES

- JSON (JavaScript Object Notation) is used to store data
- in key-value pair format.
- We use json module to read and write JSON files.

**Example:**

```
import json
data = {"name": "Ronak", "age": 22, "skills": ["Python", "ML"]}
with open("data.json", "w") as f:
    json.dump(data, f, indent=4)
with open("data.json", "r") as f:
    new_data = json.load(f)
print(new_data["name"], new_data["skills"])
```

**Output:**

```
Ronak
['Python', 'ML']
```

## 4. EXCEPTION HANDLING WITH FILES

- Useful to handle file related errors like file not found, permission denied etc.

**Example:**

```
try:
    with open("sample.txt", "r") as f:
        print(f.read())
except FileNotFoundError:
    print("File not found")
except PermissionError:
    print("Permission denied")
finally:
    print("Execution completed")
```

**Output (if file not exists):**

```
File not found
Execution completed
```

## 5. SPECIAL FILE METHODS

| Method | Description |
|---|---|
| read() | Read entire file |
| readline() | Read one line |
| readlines() | Read all lines (list) |
| write() | Write data to file |
| writelines() | Write a list of lines |
| seek() | Move file pointer to a position |
| tell() | Return current file pointer position |
| truncate() | Remove content from a file |
| close() | Close the file |

## 6. DIRECTORY OPERATIONS USING os MODULE

| Function | Description |
|---|---|
| os.getcwd() | Get current working directory |
| os.chdir(path) | Change current directory |
| os.mkdir(path) | Create a new directory |
| os.makedirs(path) | Create nested directories |
| os.listdir(path) | List files and folders |
| os.remove(file) | Delete a file |
| os.rmdir(path) | Remove an empty directory |
| os.rename(src, dst) | Rename a file or folder |
| os.path.exists(path) | Check if path exists |
| os.path.join(a, b) | Join paths safely |

## 7. PATHLIB MODULE

- Modern and more user-friendly way to work with file paths.
- Helps in cross-platform path handling.

**Example:**

```
from pathlib import Path
p = Path("data/example.txt")
print(p.exists())      # check if file exists
print(p.name)          # file name
print(p.parent)        # parent folder
print(p.suffix)        # file extension
```

**Output:**

```
True
example.txt
data
.txt
```

## 8. WORKING WITH TEXT FILES (ADVANCED)

- We can use with statement for automatic closing.
- We can append data to a file instead of overwriting.

**Example: Append data to a file**

```
with open("log.txt", "a") as f:
    f.write("New log entry\n")
with open("log.txt", "r") as f:
    print(f.read())
```

**Output:**

```
New log entry
```

## 9. FILE HANDLING TIPS

- ✔ Always use with statement.
- ✔ Use appropriate file modes ('r', 'w', 'a').
- ✔ Handle exceptions to avoid program crash.
- ✔ Use pathlib for easier and safer path handling.
- ✔ Close files automatically using with.
- ✔ Use csv for tabular data.
- ✔ Use json for structured data.
- ✔ Check if file or directory exists before operations.

## 10. KEY POINTS

- ✔ Python provides os and pathlib for file and directory operations.
- ✔ We can read, write, append and delete files.
- ✔ CSV and JSON are commonly used data formats.
- ✔ Exception handling helps in safe file operations.
- ✔ pathlib is modern and recommended.
- ✔ File handling is important for real world projects (data processing, automation, logging, ML datasets etc.).